

1. Écrire à l'écran

Une cartouche qui affiche un mot, et rien d'autre.

1. Ce qu'il faut comprendre

Le plus petit programme qui fait quelque chose. Comme tout programme C++, il commence à `int main()` : c'est là que la console arrive.

`texte(colonne, ligne, "...")` écrit à partir d'une case : l'écran en fait **20 de large sur 18 de haut**, et la case (0, 0) est en haut à gauche.

`while (true) image();` n'est pas décoratif. Un programme qui se termine laisse le processeur partir n'importe où ; ici, il attend l'image suivante, soixante fois par seconde, et l'écran reste affiché.

2. Le programme, morceau par morceau

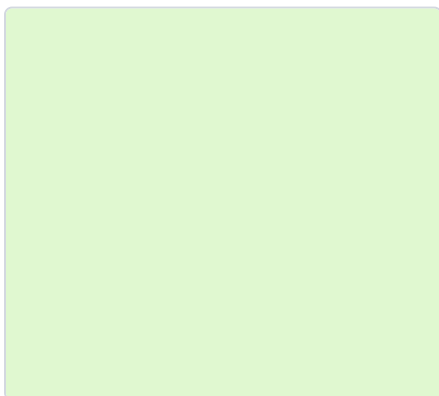
Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1291 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(5, 6, "BONJOUR");  
  
    while (true) image();  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« BONJOUR » à la colonne 5, ligne 6.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ « BONJOUR » est écrit en (5, 6)

✓ l'écran est allumé

Ensuite : **1.1. Effacer avec des espaces** — niveau 1 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++